МИРЭА- Российский технологический университет

Институт перспективных технологий и индустриального программирования

Дисциплина

«Алгоритмы машинного обучения для решения прикладных задач»

Семестр 1

Практическое занятие №5-6

Практическая работа логистическая регрессия. Практическая работа переобучение и регуляризация.

Доцент кафедры индустриального программирования
к. ф.-м. н.
Осипов Алексей Викторович

Москва, 2026 г.

Практическая работа: "Знакомство с алгоритма k-ближайших соседей (kNN)"

K-ближайший сосед с использованием scikit-learn

Задача обучения

Набор данных "Ирисы Фишера". Нужно определить к какому классу принадлежит неопознанное растение ириса, Классификация осуществляется на основе размеров чашелистиков и лепестков.

In [8]:
```
#Импорт необходимых библиотек
import warnings

warnings.filterwarnings('ignore')
```

In [50]:
```
#!pip install mlxtend
```

In [26]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import itertools
from mlxtend.plotting import plot_decision_regions

from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
#from sklearn.metrics import classification_report, accuracy_score,  plot_confusion_matrix, confusion_matrix
from sklearn.metrics import classification_report, accuracy_score,  ConfusionMatrixDisplay, confusion_matrix
%matplotlib inline
plt.rcParams["figure.figsize"] = (10,5)
```

Изучение набора данных

Набор данных "Ирисы Фишера" — это популярный многомерный набор данных, часто используемый в машинном обучении. Набор данных состоит из 150 образцов, равномерно распределенных по трем видам ирисов: ирис сетозный, ирис виргинский,

ирис разноцветный. Каждый образец состоит из четырех атрибутов: длины и ширины чашелистиков и лепестков в сантиметрах. Задача состоит в том, чтобы предсказать, к какому классу принадлежит неопознанное растение ириса, основываясь на размерах чашелистиков и лепестков.

In [27]:
```
dataset = datasets.load_iris()
df = pd.DataFrame(data=np.c_[dataset['data'], dataset['target']],
                  columns=dataset['feature_names'] + ['target'])

df.head()
```

Out[27]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0

Описания наборов данных

In [28]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    float64
dtypes: float64(5)
memory usage: 6.0 KB
```

In [31]:
```
dict(zip( dataset.target_names, df['target'].unique()))
```

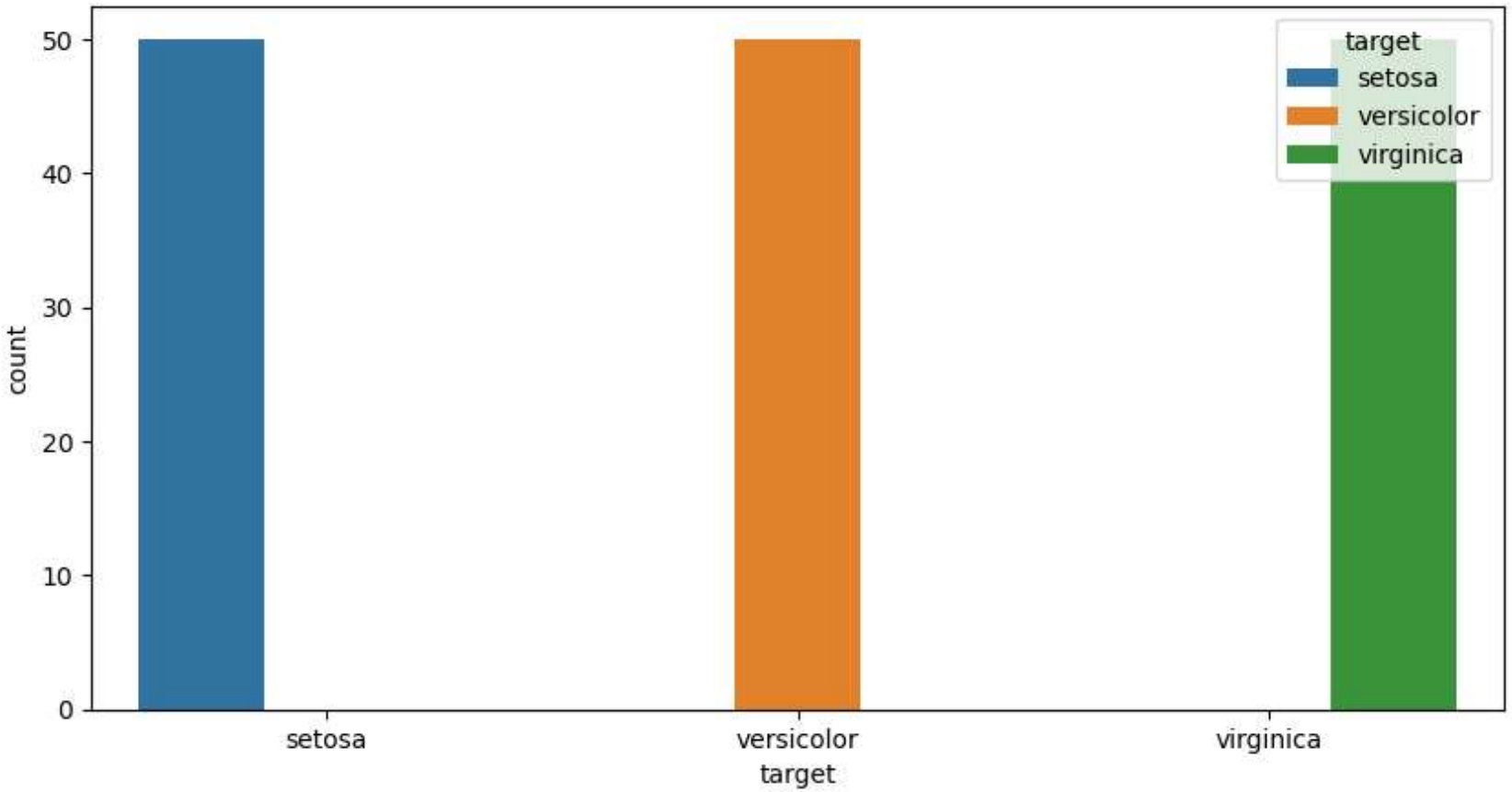
Out[31]:
```
{'setosa': 0.0, 'versicolor': 1.0, 'virginica': 2.0}
```

In [32]:
```
temp_df = df.copy()

temp_df['target'] = temp_df['target'].map(dict(zip( temp_df['target'].unique(), dataset.target_names)))
```

In [33]:
```
sns.countplot(x='target', hue='target', data=temp_df)
```

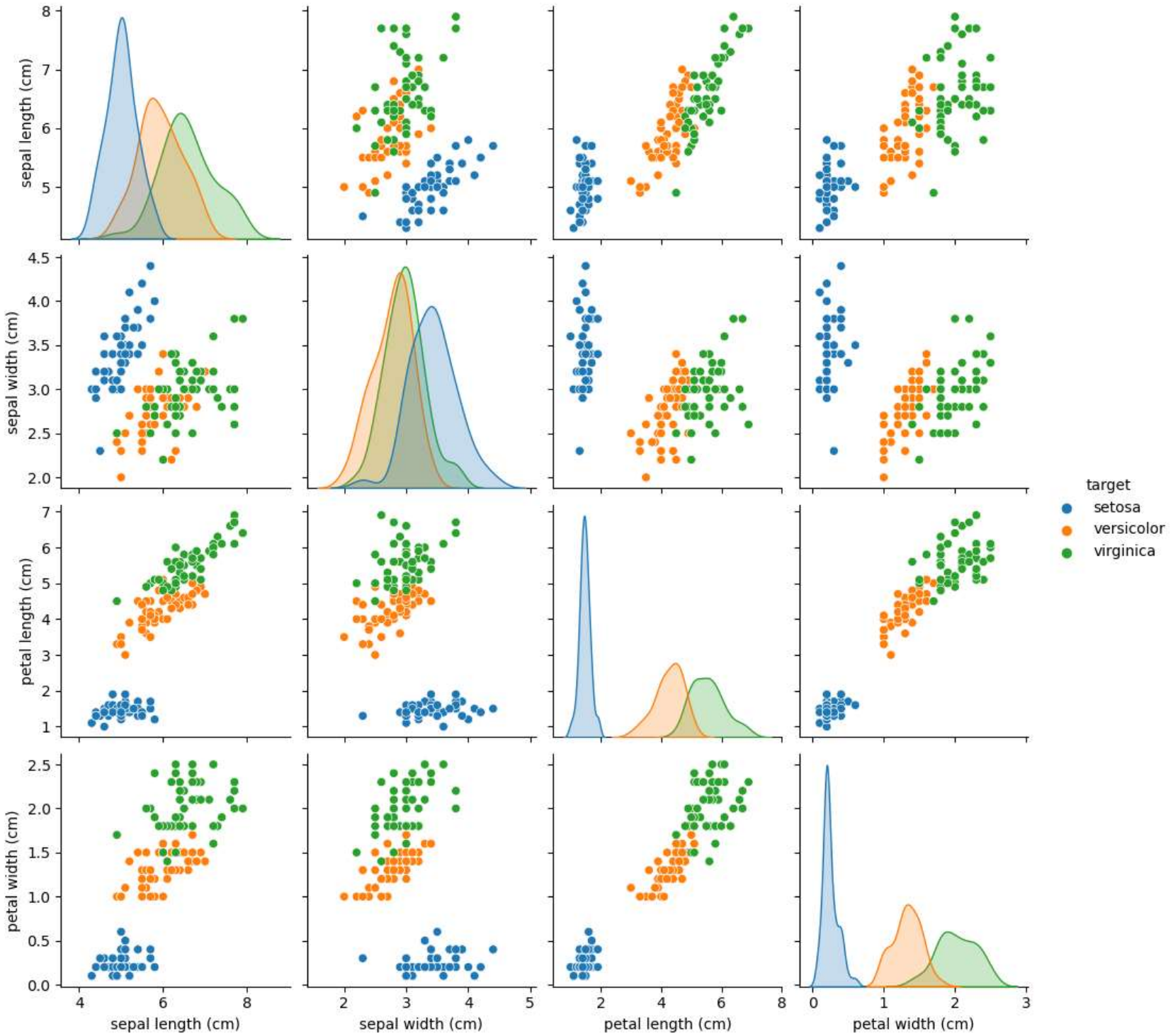
Out[33]:
```
<Axes: xlabel='target', ylabel='count'>
```



Из графика выше мы видим, что наши данные идеально сбалансированы.

In [34]:
```
sns.pairplot(temp_df, hue='target')
```

Out[34]:
```
<seaborn.axisgrid.PairGrid at 0x28181eb31d0>
```


sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
target
setosa
versicolor
virginica

На графике выше мы можем видеть взаимосвязь между каждой функцией. Мы можем увидеть, являются ли признаки разделимыми или нет.

Синтетический набор данных KNN

In [35]:
```
from IPython.display import display, Javascript, HTML
from IPython.display import IFrame
display(HTML('build.html'))
```

build.html

Модели KNN

Теперь мы построим разные модели для разных пар признаков, чтобы увидеть, насколько полезны признаки для определения их вида.

In [36]:
```
# Helper functions

def display_confusion_matrix(
    matrix, labels=None,
    title="Confusion Matrix",
    xlabel="Predicted",
    ylabel="Actual",
    figsize=None,
    cmap="Blues",
    vmin=None,
    vmax=None,
):
    """Отобразить матрицу ошибок с помощью matplotlib.

    Parameters
    ----------
    matrix : array-like
         Матрица ошибок для отображения, размерностью [n_classes, n_classes].

    labels : array-like, default=None
         Метки классов для матрицы, размерность [n_classes].

    title : str, default='Confusion Matrix'
    xlabel : str, default='Predicted'
    ylabel : str, default='Actual'
        Метки для отображения на матрице ошибок (титульная часть, абсцисса и ордината).

    figsize : array-like
        Определяет размер изображения.

    cmap : matplotib.colors.Colormap or str, default='Blues'
        Цветовая карта.

    vmin,vmax : int or float
          *vmin* и *vmax* определяют диапазон данных, который охватывает цветовая карта.
    """

    if labels is None:
        labels = [""]*len(matrix)
    if vmin is None:
        vmin = 0.
    if vmax is None:
        vmax = np.sum(matrix[0])
```

```
    t_hold = (vmax-vmin) / 2.
    fig,ax = plt.subplots(figsize=figsize)
    ax.imshow(matrix, cmap=cmap)
    for row,col in itertools.product(range(len(labels)), range(len(labels))):
        color = "white" if matrix[row,col] > t_hold else "black"
        ax.text(col, row, matrix[row, col], horizontalalignment="center", color=color)
    tick_marks = np.arange(len(labels))
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_xticks(np.arange(len(labels)))
    ax.set_xticklabels(labels, rotation=40, ha="right")
    ax.set_ylabel(ylabel)
    ax.set_yticks(np.arange(len(labels)))
    ax.set_yticklabels(labels)
    plt.show()
```

Модель 1 (длина чашелистика и ширина чашелистика)

Для начала мы собираемся построить модель KNN с длиной и шириной чашелистика. Мы обучим модель этим двум функциям и оценим ее эффективность с помощью карты классификации и матрицы путаницы.

In [37]:
```
X = df[['sepal length (cm)', 'sepal width (cm)']]
y_true = df['target']
model_1 = KNeighborsClassifier()
model_1.fit(X, y_true)
```

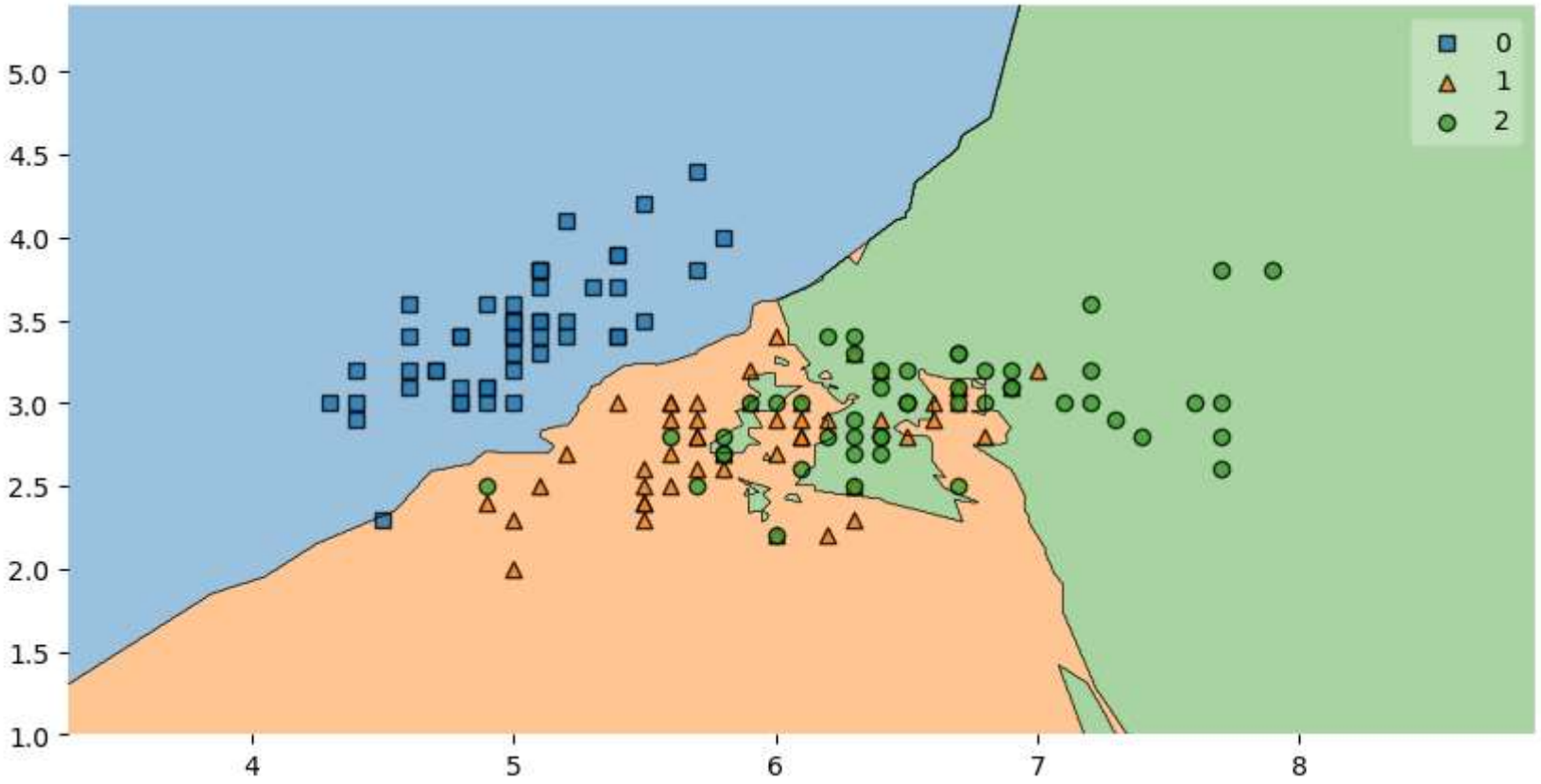
Out[37]:
```
KNeighborsClassifier()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Оценка модели 1

In [38]:
```
plot_decision_regions(X.values, y_true.values.astype(np.integer), model_1)
```

Out[38]: <Axes: >

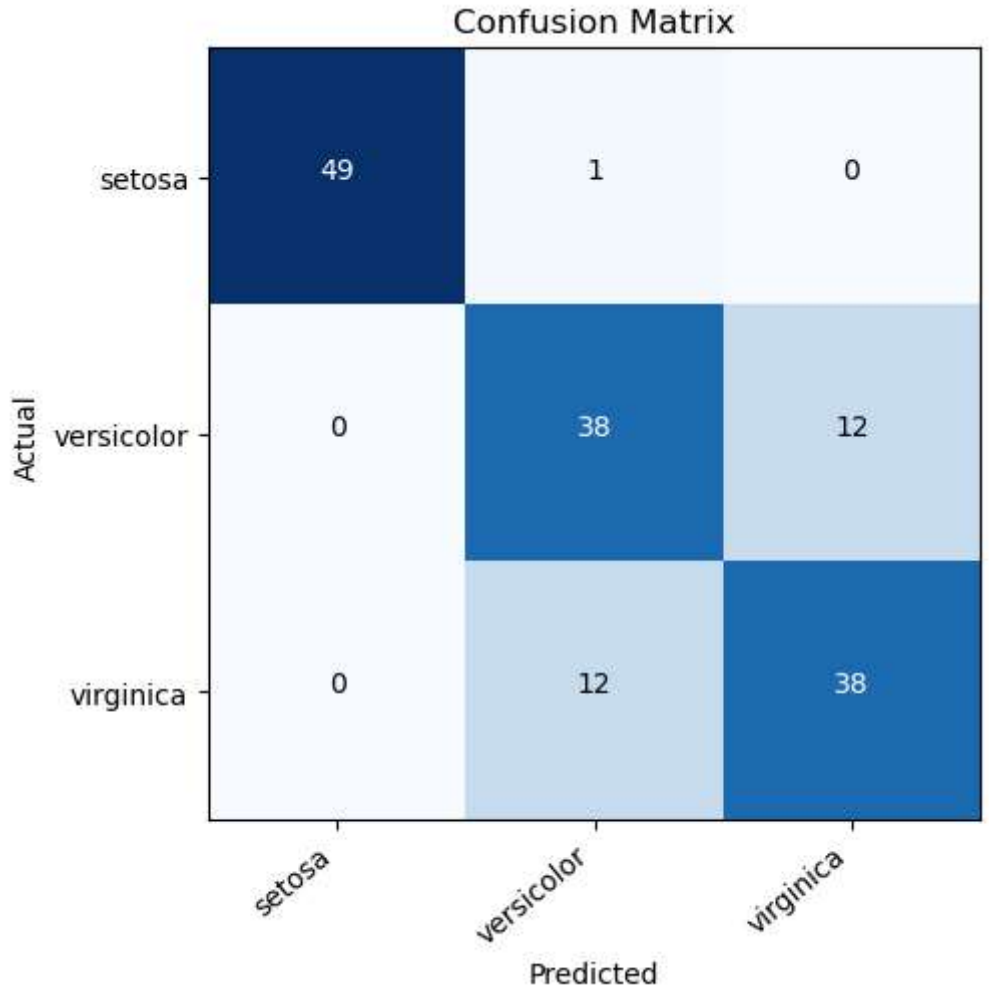


In [39]:
```
y_hat = model_1.predict(X)
labels = df["target"].unique()

matrix = confusion_matrix(y_true, y_hat, labels=labels, normalize=None)
print(matrix)

display_confusion_matrix(matrix, dataset.target_names)
```

```
[[49  1  0]
 [ 0 38 12]
 [ 0 12 38]]
```



Модель 2 (длина и ширина лепестка)

Теперь для нашей второй модели мы собираемся использовать функции длины и ширины лепестка в качестве входных данных.

In [40]:
```
X = df[['petal length (cm)', 'petal width (cm)']]

model_2 = KNeighborsClassifier()
model_2.fit(X, y_true)
```

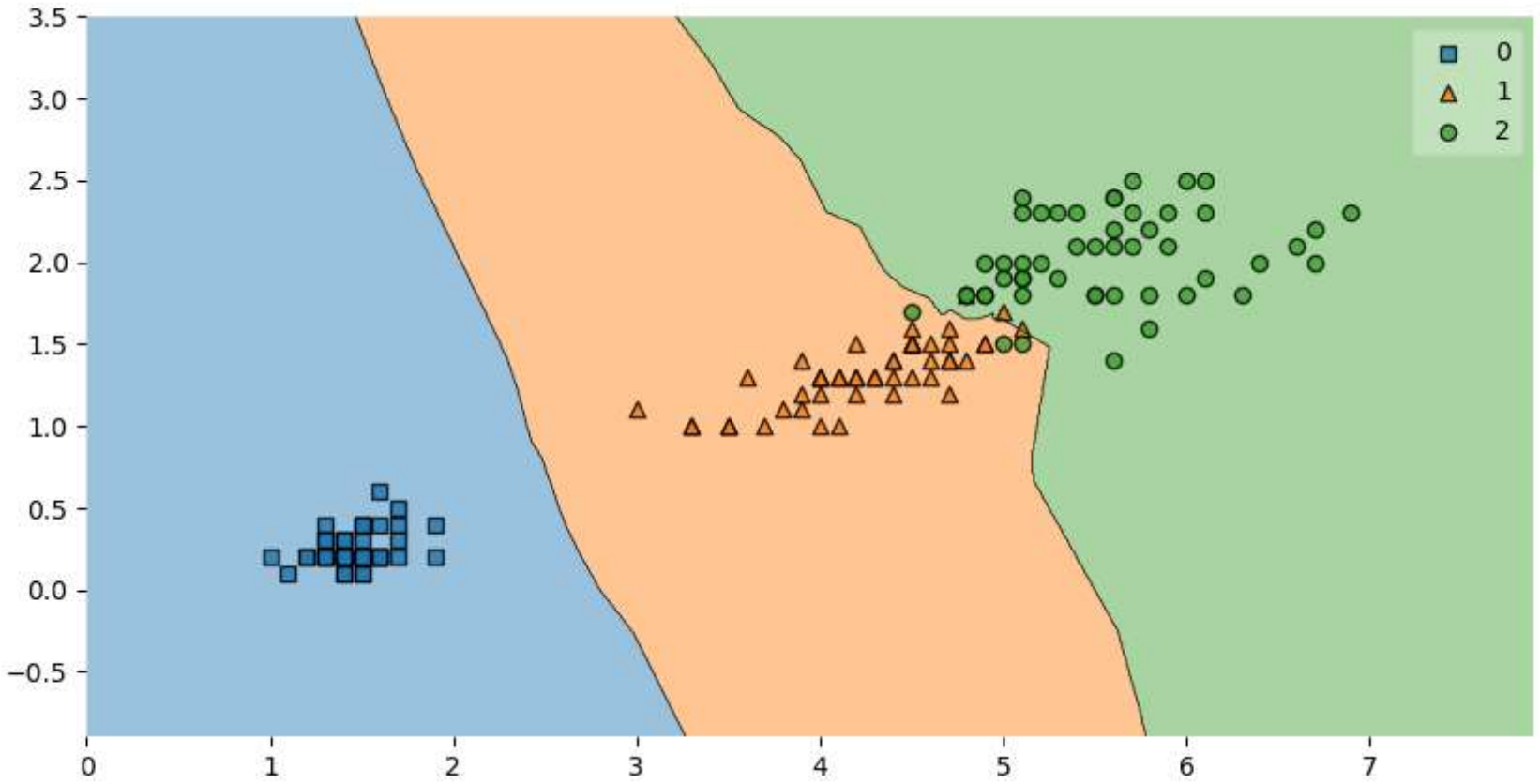
Out[40]:
```
KNeighborsClassifier()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Model 2 Evaluation

In [41]:
```
plot_decision_regions(X.values, y_true.values.astype(np.integer), model_2)
```

Out[41]: <Axes: >

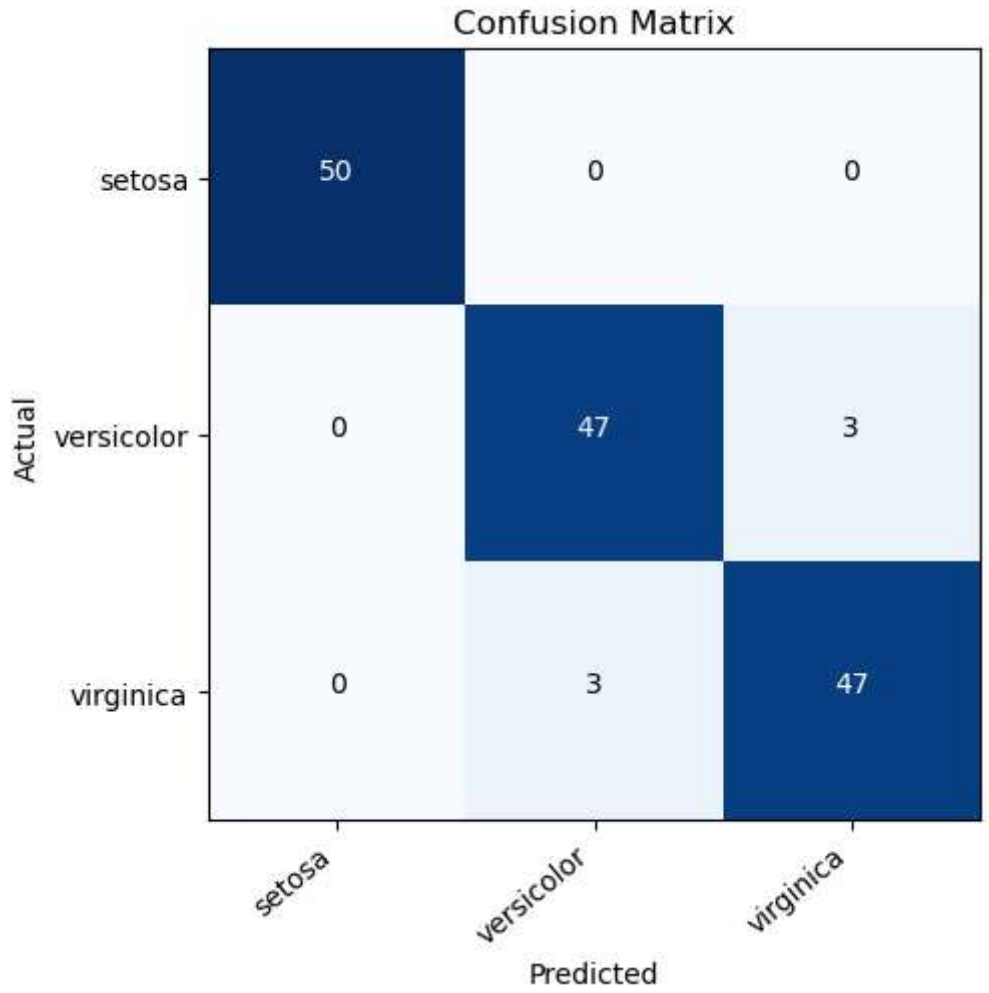


In [42]:
```
y_hat = model_2.predict(X)

matrix = confusion_matrix(y_true, y_hat, labels=labels, normalize=None)
print(matrix)

display_confusion_matrix(matrix, dataset.target_names)
```

```
[[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```



Модель 3 (все функции)

Теперь что касается нашей окончательной модели, мы собираемся использовать все доступные функции и обучить нашу модель KNN.

In [43]:
```
X = df.drop(['target'], axis=1)
```

In [44]:
```
model_3 = KNeighborsClassifier()
model_3.fit(X, y_true)
```

Out[44]:
```
KNeighborsClassifier()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

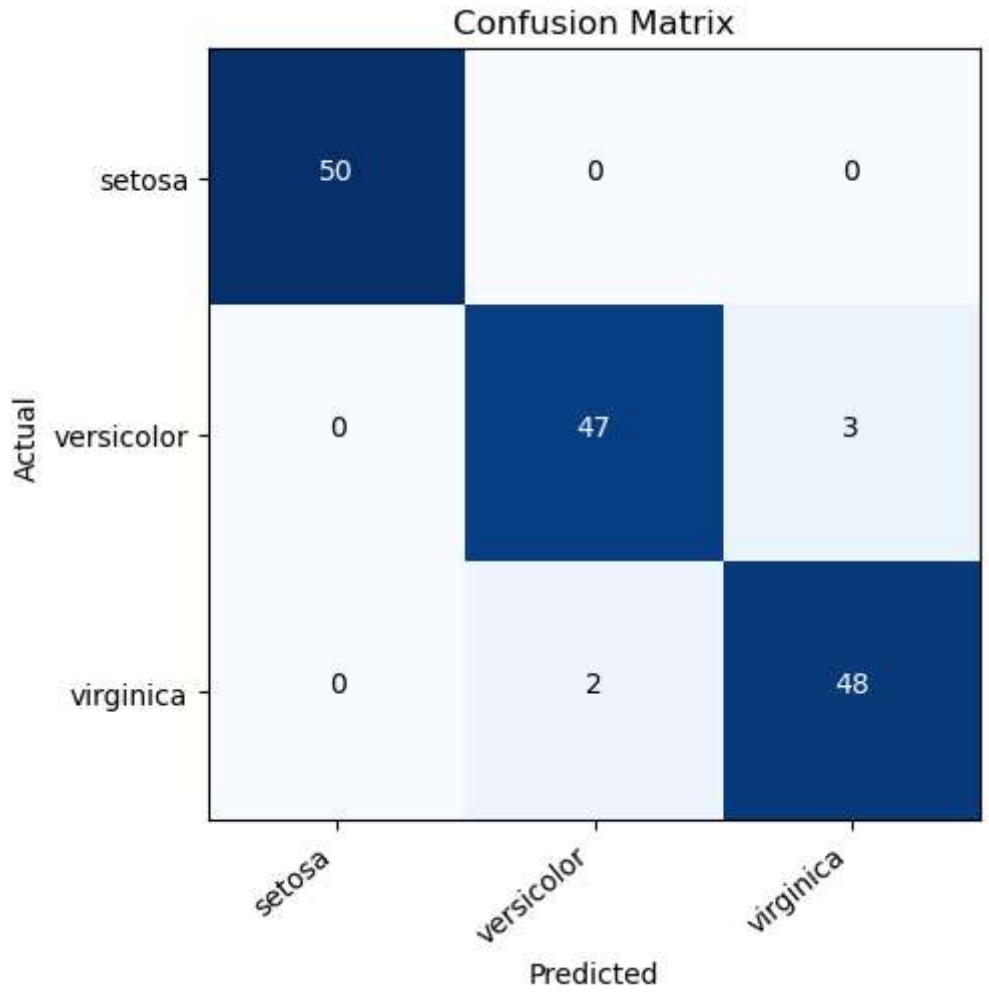
Модель 3 Оценка

In [45]:
```
y_hat = model_3.predict(X)

matrix = confusion_matrix(y_true, y_hat, labels=labels, normalize=None)
print(matrix)

display_confusion_matrix(matrix, dataset.target_names)
```

```
[[50  0  0]
 [ 0 47  3]
 [ 0  2 48]]
```



Вы можете видеть, что окончательная модель со всеми функциями работает лучше, чем все другие модели. Меньше ошибочной классификации.

Однако до сих пор мы обучали и оценивали наши модели в одном и том же наборе данных. Использование одних и тех же данных для обучения и оценки не рекомендуется.

Модель на разделенном наборе данных

Здесь мы разделим наш набор данных на обучающий и тестовый набор и обучим нашу модель в обучающем наборе. Позже во время оценки мы собираемся использовать наш изолированный набор тестов.

In [46]:
```
X = df.drop(['target'], axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.3)
```

In [47]:
```
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
```

Out[47]:
```
KNeighborsClassifier()
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [48]:
```
preds = knn.predict(X_test)

print(classification_report(preds, y_test))
```

```
              precision    recall  f1-score   support

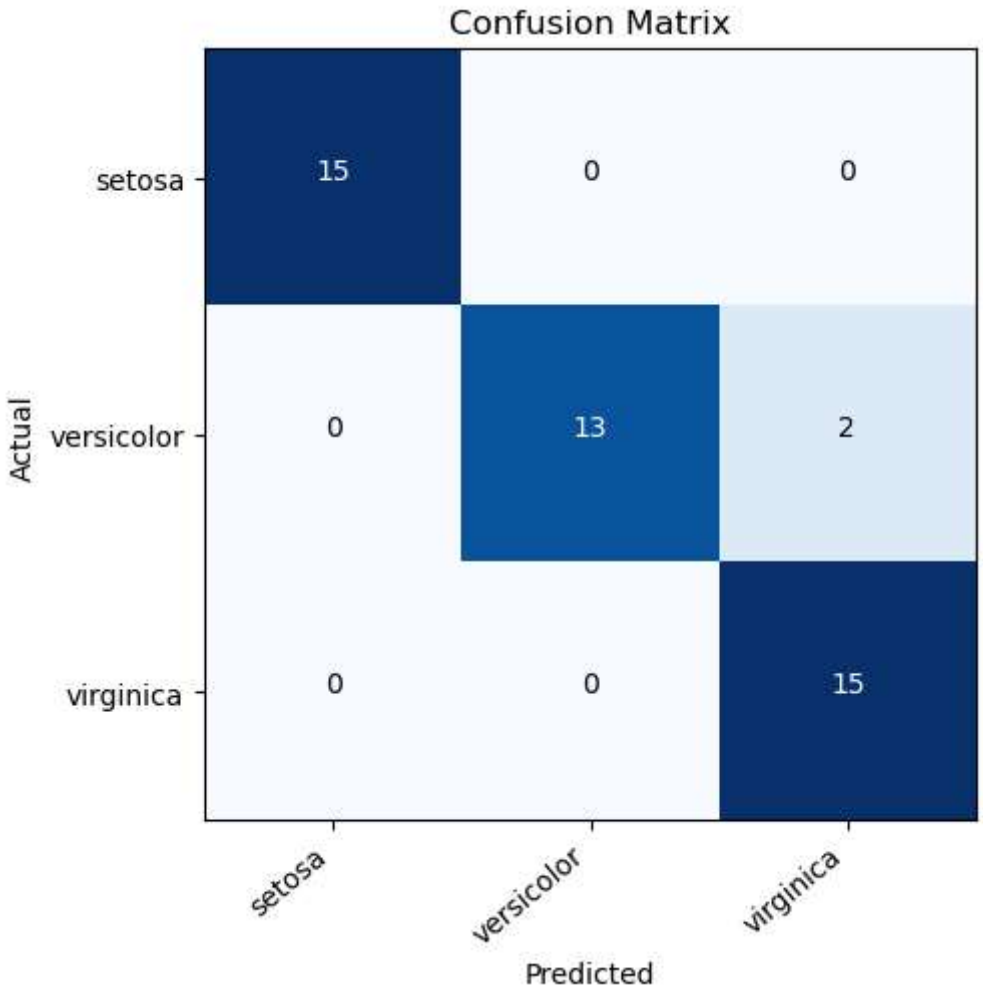
         0.0       1.00      1.00      1.00        15
         1.0       0.87      1.00      0.93        13
         2.0       1.00      0.88      0.94        17

    accuracy                           0.96        45
   macro avg       0.96      0.96      0.96        45
weighted avg       0.96      0.96      0.96        45
```

In [49]:
```
matrix = confusion_matrix(y_test, preds, labels=y.unique(), normalize=None)
print(matrix)

display_confusion_matrix(matrix, dataset.target_names)
```

```
[[15  0  0]
 [ 0 13  2]
 [ 0  0 15]]
```



Выше вы можете видеть, что точность нашего набора тестовых данных составляет 98%.

Из матрицы путаницы мы также видим, что у нас очень низкий уровень ошибок классификации.

Для наших вышеупомянутых моделей мы также можем использовать данные обучения и тестирования отдельно, оценивать эти модели, а затем выбирать лучшую модель.

Задание

С помощью метода kNN определить качество красного вина. Использовать датасет winequality-red.csv, который находится в папке data. Для этого разделить набор данных на обучающий и тестовый набор и обучим нашу модель в обучающем наборе. Построить матрицу путаници. Сделать выводы по полученным результатам.

In [ ]: